Northwind Payments - Engineering Handbook

This handbook describes how the Northwind Payments engineering team builds, ships and operates the payment platform. Northwind processes card, bank transfer and instant payments for small online merchants. The platform is a set of Kotlin and Python services that communicate over Kafka and REST, deployed on AWS ECS in two regions. Every service has an owning team, a runbook and a dashboard. New engineers should read the sections on deployments, on-call and incident response during their first week.

1. Deployments

All services deploy through the CI pipeline; manual deploys to production are not allowed. A change reaches production in three stages: staging, a canary that receives five percent of traffic for thirty minutes, and then the full fleet. The canary is rolled back automatically when the error rate rises above one percent or the p99 latency doubles. Deploys are frozen from Friday 16:00 to Monday 09:00 and during the last three business days of each month, when merchants run their billing cycles. Risky changes to payment flows must ship behind a feature flag so they can be turned off without a deploy.

2. On-call

Each team runs a weekly on-call rotation that starts on Wednesday at 10:00. The primary on-call engineer must acknowledge a page within five minutes and start working on it within fifteen minutes. A secondary engineer is paged automatically if the primary does not acknowledge in time. On-call engineers receive a compensation day for every weekend they are on call. Handover happens in a fifteen-minute meeting where the outgoing engineer reviews open alerts, recent incidents and anything unusual in the dashboards.

3. Incident response

An incident is any event that affects merchants' ability to receive payments. Severity one means payments are failing for more than ten percent of merchants; severity two means a single large merchant or a single payment method is affected; severity three is degraded performance without failed payments. For severity one and two, the on-call engineer opens an incident channel, names an incident commander and posts updates every thirty minutes. A blameless postmortem is written within five business days for every severity one or two incident, with action items tracked to completion.

4. Refunds API

Merchants create refunds with POST /v2/refunds, passing the payment id and an amount. Partial refunds are allowed until the refunded total equals the captured amount. Card refunds can be requested up to one hundred eighty days after capture; instant payment refunds up to ninety days. The API is idempotent: clients send an Idempotency-Key header, and a repeated key within twenty-four hours returns the original response instead of creating a second refund. Refunds are processed asynchronously and the merchant receives a refund.succeeded or refund.failed webhook.

5. Rate limits and retries

The public API allows one hundred requests per second per merchant, with bursts up to two hundred. Requests above the limit receive HTTP 429 with a Retry-After header. Internal services retry failed calls at most three times with exponential backoff starting at two hundred milliseconds and full jitter. Calls that move money are never retried without an idempotency key. Webhooks are retried for up to three days, with the interval growing from one minute to six hours.

6. Data and privacy

Card numbers never leave the tokenization service; every other service stores only tokens. Logs must not contain card numbers, document numbers or full email addresses, and a log scanner blocks deploys that print them. Merchant data is kept for five years after the contract ends, as required by financial regulation, and then deleted. Engineers get production database access only through a break-glass request that is approved by a second engineer and expires after four hours.